

Graph Databases

1. Overview

Graph databases are databases specifically designed to **store, query, and traverse relationships between entities efficiently**.

Instead of rows/tables, they model data as:

- **Nodes** = entities
- **Edges / Relationships** = connections between entities
- **Properties** = attributes on nodes or edges

Example:

None

Samarth --FRIENDS_WITH--> Ansh

Ansh --WORKS_AT-----> Stripe

Samarth --LIVES_IN-----> Surat

2. Why Graph Databases Exist

Traditional SQL databases can represent relationships using joins, but deep multi-hop relationships become expensive and complex.

Graph DBs optimize:

- Relationship traversal
- Multi-hop queries
- Connected data
- Pattern matching
- Network analysis

3. Core Components

Component	Meaning
Node	Entity/object
Edge	Relationship
Direction	From → To
Property	Metadata

4. Example Model

None

(User: Samarth)

(User: Ansh)

(Company: Stripe)

Samarth -> FOLLOWS -> Ansh

Ansh -> WORKS_AT -> Stripe

5. Popular Examples

- Neo4j
- RedisGraph
- Amazon Neptune
- JanusGraph

6. Best Use Cases

Use graph DBs when relationships are core to product logic:

- Social networks
- Friend recommendations
- Fraud detection
- Knowledge graphs
- Supply chain dependencies
- Network topology
- Recommendation engines
- Identity resolution

7. Example Queries

Friends of Friends

None

Find people connected to Samarth within 2 hops

Recommendation

None

Users who bought X also bought Y

Fraud Ring

None

Multiple accounts using same phone/address/device

8. Why Better Than SQL for Connected Data

In SQL:

None

Users JOIN Friends JOIN Friends JOIN Purchases JOIN Devices...

Gets complex and slow for many hops.

In graph DB:

Traversal is natural and optimized.

9. Graph Data Model Example

Nodes

None

User

Product

City

Company

Device

Relationships

None

BOUGHT

LIVES_IN

FOLLOWS

USES

WORKS_AT

REFERRED

10. Real-World Example: Social Media

None

A follows B

B follows C

C likes D

Queries:

- Suggested follows
- Mutual connections
- Influence paths
- Community detection

11. Real-World Example: Fraud Detection

None

Account1 -> uses -> DeviceX

Account2 -> uses -> DeviceX

Account3 -> uses -> DeviceX

This reveals suspicious linked accounts quickly.

12. Query Languages

Different graph DBs use graph query languages:

- Cypher
- Gremlin
- SPARQL (RDF graphs)

Example concept:

None

```
MATCH (u)-[:FOLLOWS]->(x)
```

```
RETURN x
```

13. Graph DB vs SQL

Feature	SQL	Graph DB
Transactions	Strong	Varies
Tabular data	Excellent	Moderate
Multi-hop relationships	Complex	Excellent

Deep joins	Slower	Fast traversal
Aggregations	Strong	Moderate

14. Graph DB vs Document DB

Feature	Document DB	Graph DB
Nested objects	Excellent	Good
Relationship-heavy data	Moderate	Excellent
Traversal queries	Weak	Strong

15. Scaling Considerations

Graph databases can be harder to shard than key-value systems because relationships often span partitions.

Need planning for:

- Partitioning graph data
- High-degree nodes
- Hotspot users
- Traversal latency

16. Trade-offs

Not ideal for simple CRUD apps

Complex distributed scaling

Smaller ecosystem than SQL

Can be overkill if relationships simple

17. Interview Answer Example

Q: Why choose Neo4j?

I'd use Neo4j because relationships are central to the system, and we need fast multi-hop traversal for recommendations and fraud detection.

18. Short Revision Notes

None

Graph DB:

- Nodes
- Edges
- Properties

Best for:

social network

fraud detection

recommendation

knowledge graph

Examples:

Neo4j, Neptune

19. One-Line Summary

Graph databases model data as connected nodes and relationships, making them ideal for systems where querying and traversing relationships is the primary challenge.